

Artificial Intelligence for robotics

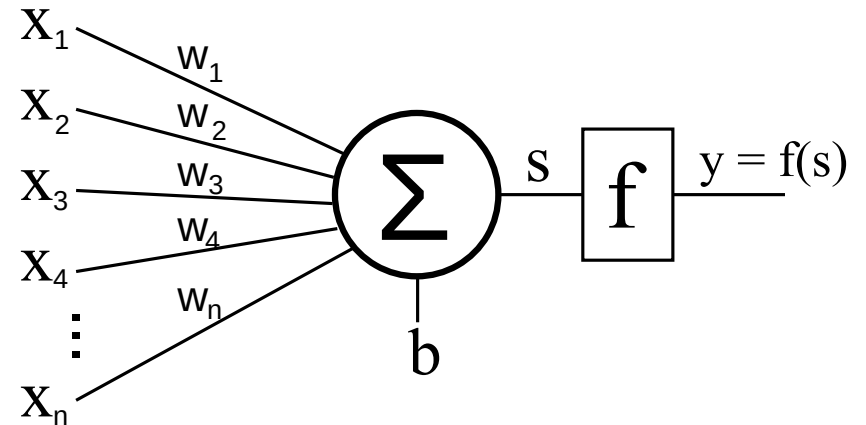
Part 4 : Single-layer neural networks

Simon Gay

Artificial Intelligence for Robotics

- **Formal Neuron**

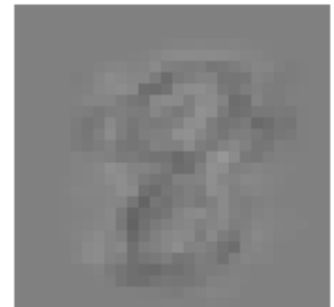
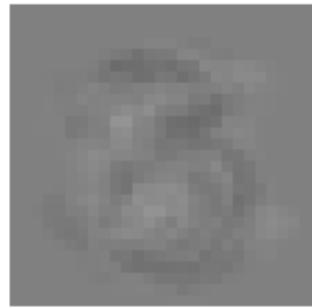
- A set of weights
- Learning based on examples
- Widrow-Hoff rule
- Good recognition rate



Artificial Intelligence for Robotics

- **Formal Neuron**

- The formal neuron produces an “average” image of positive and negative results, highlighting characteristic elements allowing the recognition, even on unknown images

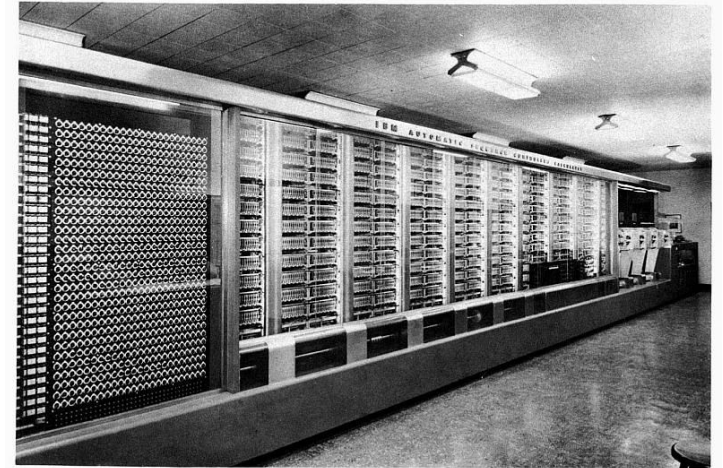


- **But WHY does this work ?**

Artificial Intelligence for Robotics

- **The Perceptron**

- One of first supervised networks
 - Invented in 1957 by Frank Rosenblatt
 - One or more formal neurons
 - Hebb's rule
- Threshold activation function: each neuron is a binary classifier



II Front View of the Calculator

Artificial Intelligence for Robotics

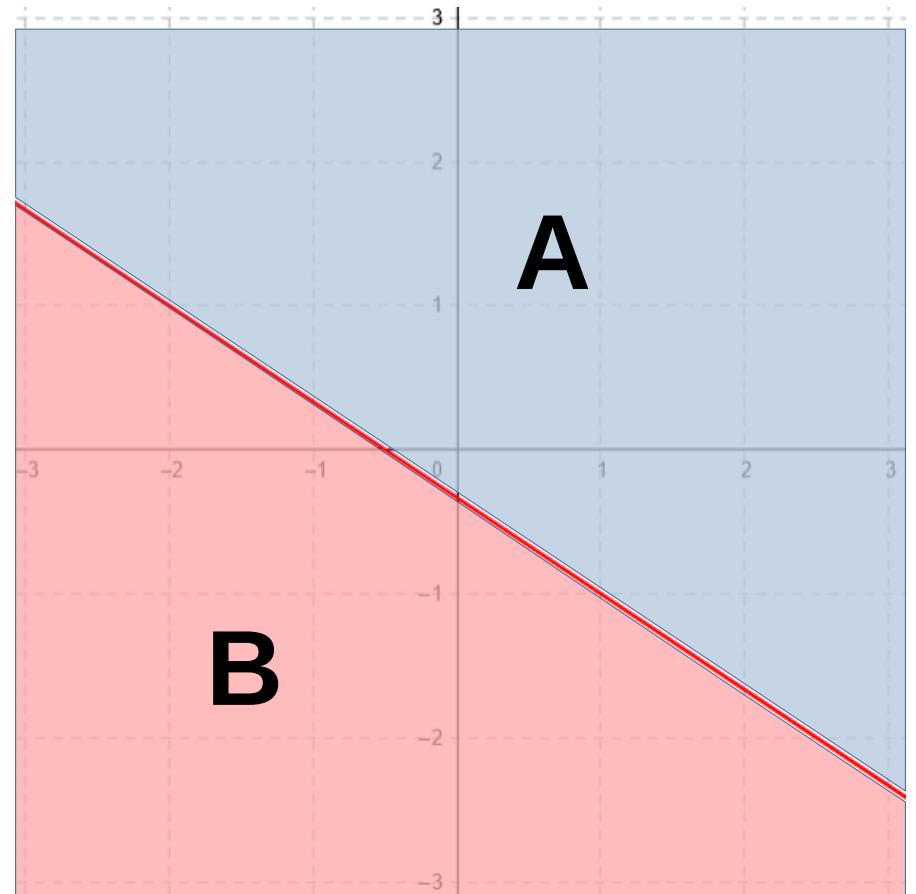
- **Linear function in a plane**

example:

$$3.y + 2.x + 1 = 0 \quad (y = -(2/3).x - 1/3)$$

This function split the plane in 2:

- If $3.y + 2.x + 1 > 0$
→ space A above the line
- If $3.y + 2.x + 1 < 0$
→ space B below the line



Artificial Intelligence for Robotics

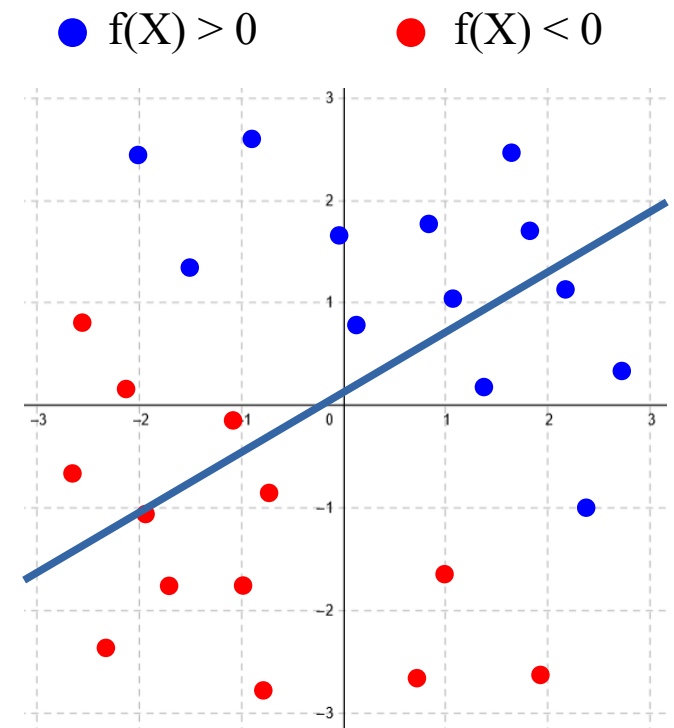
- **And if we do not know the function parameters ?**

We have an unknown function $a.x + b.y + c = 0$

and a set of points $X_i = [x_i, y_i]$ such as $f(X_i) > 0$ or $f(X_i) < 0$

→ We must find a triplet (a, b, c) matching all constraints

- Select a random triplet
- Each sample is tested
 - If $f(X_i) > 0$ but $ax_i + by_i + c < 0$
 - $ax_i + by_i + c$ should be increased
 - If $f(X_i) < 0$ but $ax_i + by_i + c > 0$
 - $ax_i + by_i + c$ should be decreased
 - Else, nothing change



Artificial Intelligence for Robotics

- **And if we do not know the function parameters ?**

For each parameter a , b and c , the value must be slightly increased or decreased proportionally to, respectively, x , y and 1

$$a \leftarrow a + \alpha \cdot x_i \quad \text{or} \quad a \leftarrow a - \alpha \cdot x_i$$

$$b \leftarrow b + \alpha \cdot y_i \quad \text{or} \quad b \leftarrow b - \alpha \cdot y_i$$

$$c \leftarrow c + \alpha \cdot 1 \quad \text{or} \quad c \leftarrow c - \alpha \cdot 1$$

- **If we note $r_i = 1$ if $f(X_i) > 0$ and $r_i = -1$ si $f(X_i) < 0$**

– Then

- $a \leftarrow a + \alpha \cdot r_i \cdot x_i$

- $b \leftarrow b + \alpha \cdot r_i \cdot y_i$

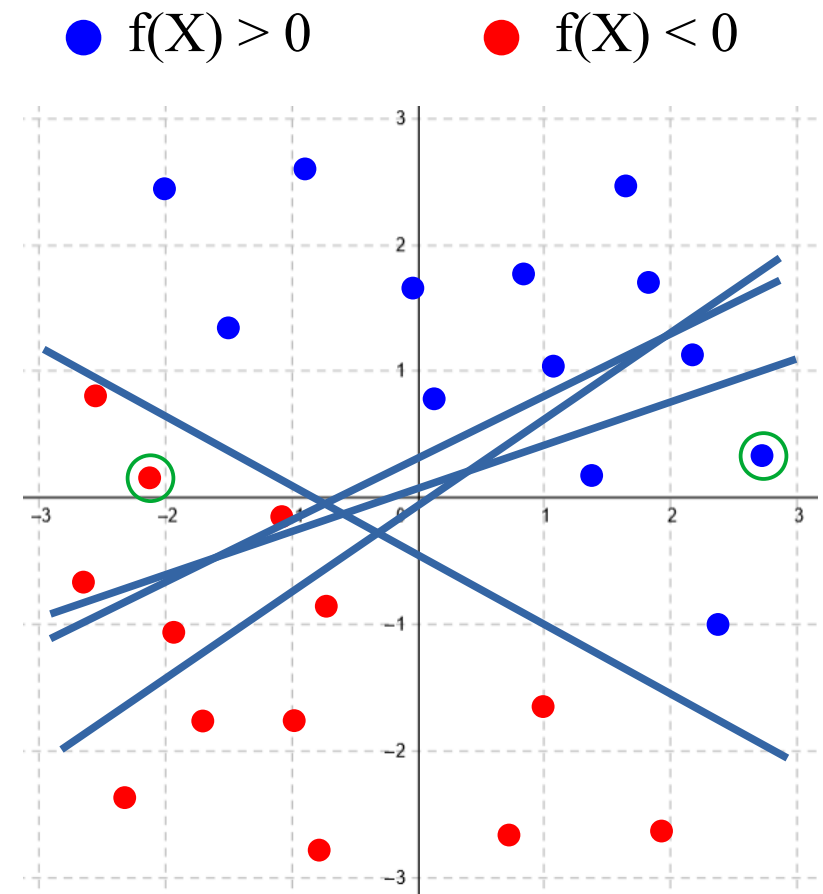
- $c \leftarrow c + \alpha \cdot r_i$

– All points X_i are tested until all of them are in the right side of the line

Artificial Intelligence for Robotics

- Parameters will converge to a solution

- Algorithm:
error = true
while error do
 error = false
 for each X_i do
 if $f(X_i) \cdot (a \cdot x_i + b \cdot y_i + c) < 0$ then
 $a += \alpha \cdot r_i \cdot x_i$
 $b += \alpha \cdot r_i \cdot y_i$
 $c += \alpha \cdot r_i$
 error = true
 endif
 endfor
endwhile



Artificial Intelligence for Robotics

- **What about Perceptron ?**

- Let's write:

$$- \quad a \cdot x + b \cdot y + c = 0 \quad \rightarrow \quad w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$$

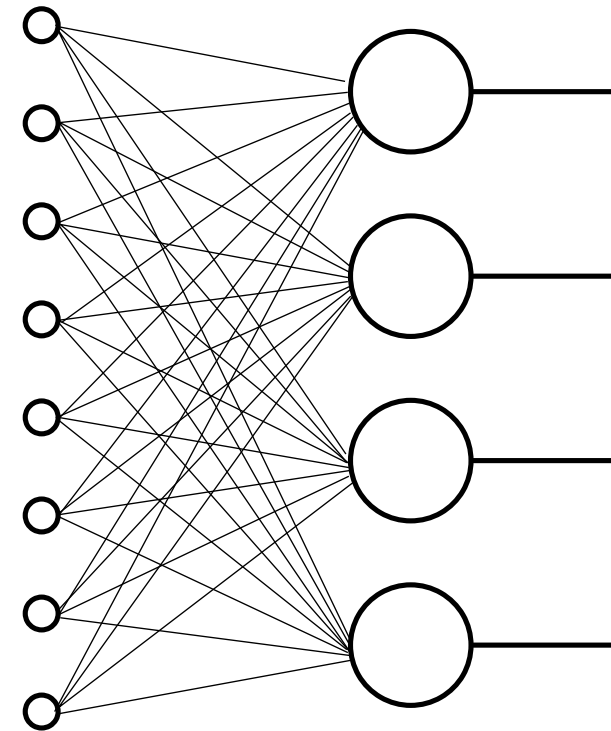
- We can generalize to a n-dimension space:

$$\sum_k w_k \cdot x_k + b = 0$$

- Weights of a formal neuron form the equation of an hyperplane
- The learning process change weights to split space in two to form two groups of points (samples of the dataset)

Artificial Intelligence for Robotics

- **Perceptron with multiple neurons**
- **Used to define more than one category**
 - Output is a binary code
 - Examples: ascii code of a letter, conversion binary to 7-segment digit...
 - One neuron per category ?
 - Not convenient as multiple neurons can have an output at 1
 - neurons with continuous activation function



Artificial Intelligence for Robotics

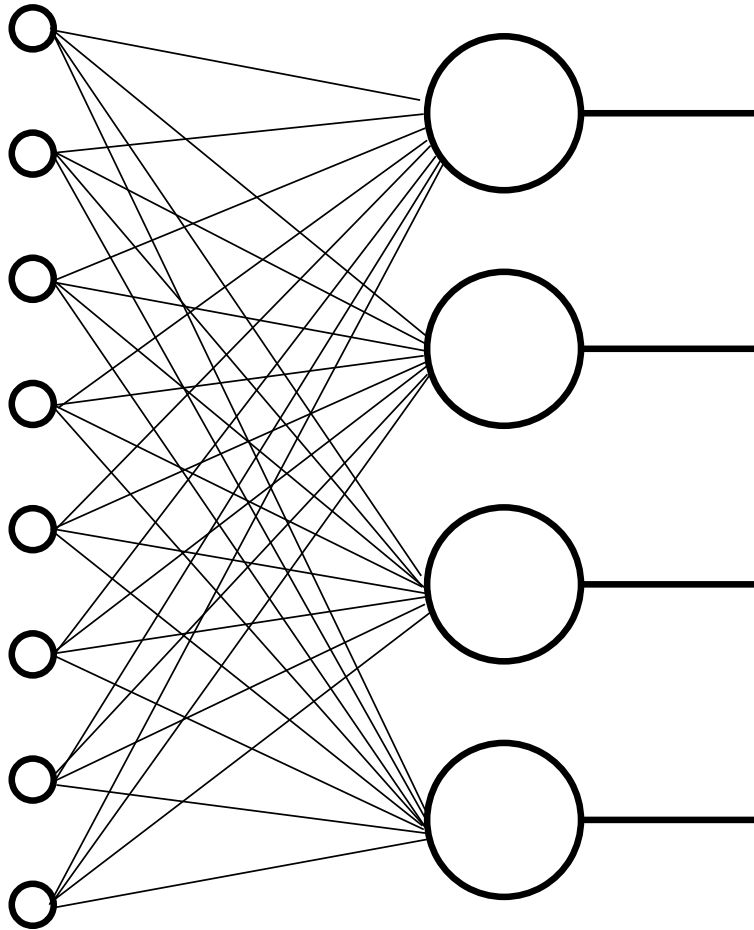
- **Perceptron with continuous activation function: single-layer network**
- Functions: sigmoid, tanh, linear, RELU ...
 - Adds a confidence level in the result
 - The closer the result is to 0 or 1, the more certain we can be of it
 - Competition between neurons: we only consider the most active neuron (no threshold)



(1 : 0,013) (2 : 0,658) (3 : 0,553) (4 : 0,350) (5 : 0,112) (6 : 0,092)...

- Learn with Widrow-Hoff rule
 - Consider the error level (continuous delta)

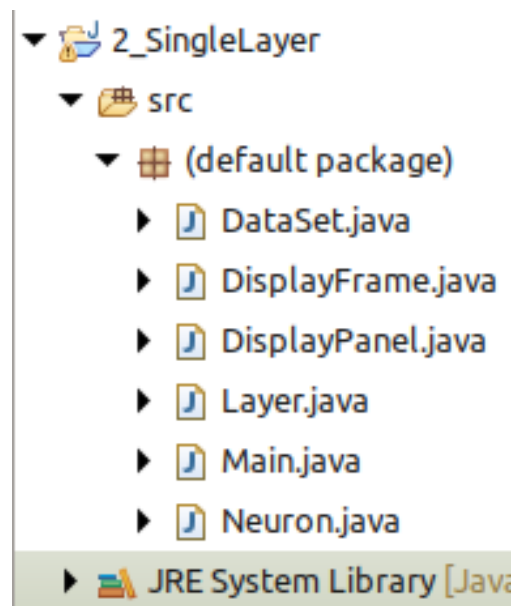
Artificial Intelligence for Robotics



Let's code !

Artificial Intelligence for Robotics

- **We will implement a single-layer network to recognize digits in images**
- Duplicate project 1_neuron and call the copy 2_SingleLayer
 - Right-click on '1_Neuron' → copy → paste in 'Package Explorer' and rename
 - Add a new class 'Layer' to the project
 - Right-click on “(default package)” then “New” → “Class” → name → “Finish”



Artificial Intelligence for Robotics

- **A single layer network is nothing more than a list of formal neurons**
- In class Layer, add:
 - a vector of Neuron called *layer*
 - a vector of float named 'results' to collect and store neurons' results
 - a float named 'sum_delta' (used to measure performances)

```
1  
2 public class Layer {  
3  
4     public Neuron[] layer;  
5     public float[] results;  
6     public float sum_delta;  
7  
8 }  
9
```

Artificial Intelligence for Robotics

- **A single layer network is nothing more than a list of formal neurons**
- Add the Layer class's constructor
 - Parameters: number of input and number of outputs (number of neurons)
 - Initialize the vectors

```
2 public class Layer {
3
4     public Neuron[] layer;
5     public float[] results;
6     public float sum_delta;
7
8     public Layer(int nb_input, int nb_output) {
9
10         layer=new Neuron[nb_output];
11         for (int i=0;i<layer.length;i++) layer[i]=new Neuron(nb_input);
12         results=new float[layer.length];
13     }
14 }
15
```

Artificial Intelligence for Robotics

- Add function to perform function compute and learn on each individual neurons
 - Be careful with input and output indexes
 - In learn function, the expected output is a vector giving the expected value of each neuron
 - The function learn must reinitialize the sum_delta value and add absolute values of delta of all neurons.

```
public Layer(int nb_input, int nb_output){  
  
    layer=new Neuron[nb_output];  
    for (int i=0;i<layer.length;i++) layer[i]=new Neuron(nb_input);  
    results=new float[layer.length];  
}  
  
public void compute(float[] input){  
    for (int i=0;i<layer.length;i++){  
        results[i]=layer[i].compute(input);  
    }  
}  
  
public void learn(float[] input, int[] output){  
    sum_delta=0;  
    for (int i=0;i<layer.length;i++){  
        layer[i].learn(input, output[i]);  
        sum_delta+=Math.abs(layer[i].delta);  
    }  
}
```

- The layer is ready !

Artificial Intelligence for Robotics

- **Integration of our layer**
- In Main class, replace the neuron pointer with a layer pointer
 - Parameters to initialize it:
 - Input: still *size_x*size_y*
 - Output: *nb_values* classes (10 digits) → replace variable “number”

```
public int epoch=0;
public int test=0;

→ public static int nb_values=10;    // number of classes

//-----
private DisplayFrame display;        // display panel

//-----
public Layer layer;                  // Neuron Layer

//////////
public Main(){

    // initialize structures
→ layer=new Layer(size_x*size_y, nb_values);

    // load test matrix
```

Artificial Intelligence for Robotics

- Integration of our layer
- Result is a one hot vector with only one element at 1 (the right answer)
- In learning loop, update the *compute* and *learn* parts
- Update sum of error (delta)

```
// dataset loop
for (test=0;test<matrixImages.length;test++){

    // define expected result
    int res=0;
    if (matrixLabels[test]==number) res=1;

    // process neuron
    neuron.compute(matrixImages[test]);
    neuron.learn(matrixImages[test], res);

    sumdelta+=Math.abs(neuron.delta); // add error (absolute value)

    display.repaint(); // update display panel
}
```



```
// dataset loop
or (test=0;test<matrixImages.length;test++){

    // define expected result
    int[] output=new int[nb_values]; // vector of 0
    output[matrixLabels[test]]=1;    // answer at 1

    // process neuron
    layer.compute(matrixImages[test]);
    layer.learn(matrixImages[test], output);

    sumdelta+=layer.sum_delta; // add error

    display.repaint(); // update display panel
    // ... (matrixLabels[test] == 10) ...
}
```

Artificial Intelligence for Robotics

- **Integration of our layer**
- In evaluation loop, the answer of the layer is the index of the neuron with greatest output → search for max value then compare index with label
- Update error count

```
// TODO : evaluation
for (test=0;test<matrixImages.length;test++){

    layer.compute(matrixImages[test]); // compute output

    int imax=0;
    float max=0;
    for (int i=0;i<layer.results.length;i++){
        if (layer.results[i]>max){
            imax=i;
            max=layer.results[i];
        }
    }

    // count errors
    if (imax!=matrixLabels[test]) errors++;
}
```

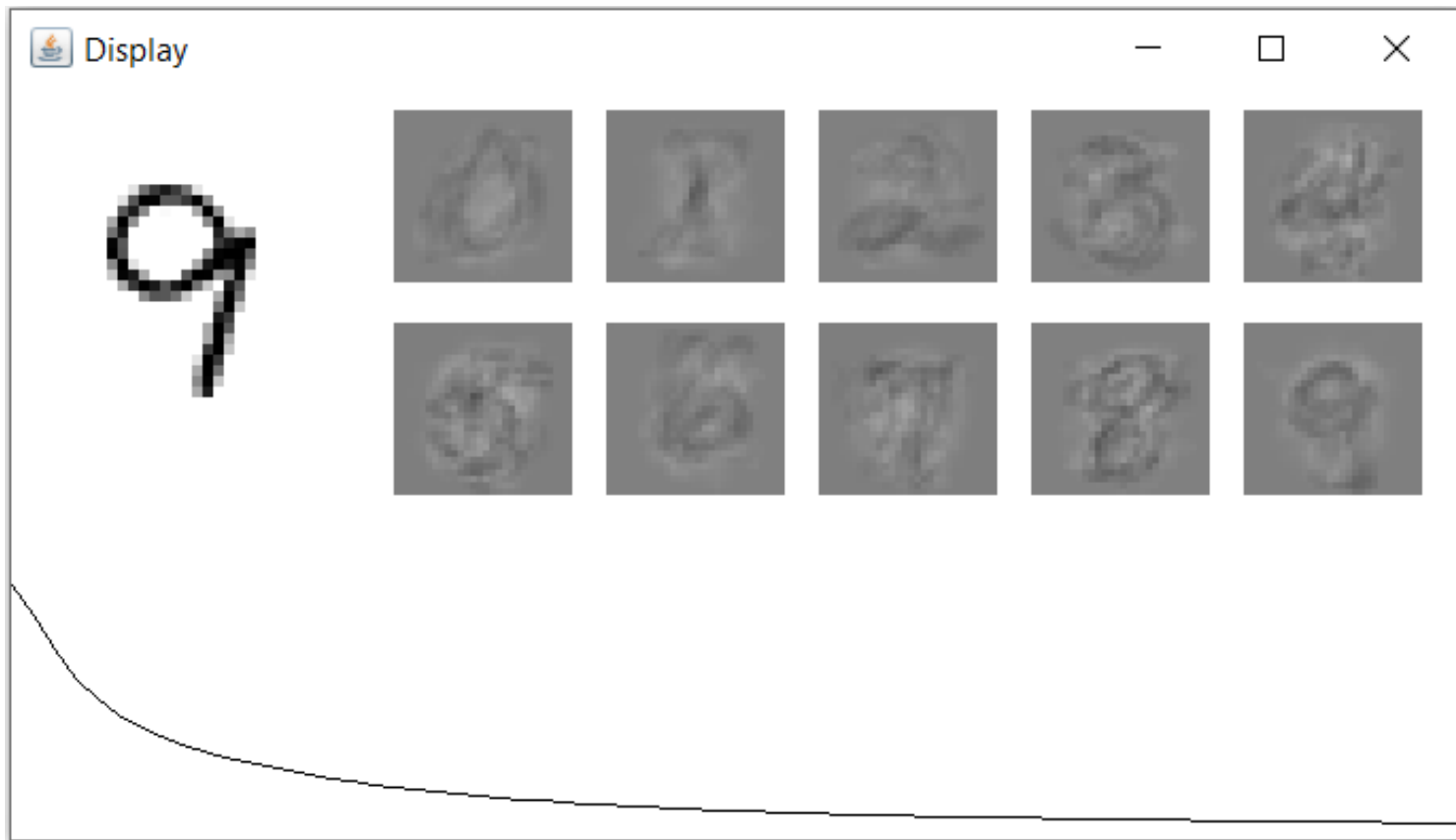
Artificial Intelligence for Robotics

- Update display panel
- We now display the weights of 10 neurons
- Increase the size of the Frame (DisplayFrame) → 700px instead of 500
- Display weights on two rows (with / and %)

```
for (int n=0;n<Main.nb_values;n++){  
    for (int i=0;i<Main.size_x;i++){  
        for (int j=0;j<Main.size_y;j++){  
            val=(int) (main.layer.layer[n].weights[i+Main.size_x*j]*50)+128;  
            if (val<0) val=0;  
            if (val>255) val=255;  
            g.setColor(new Color(val,val,val));  
            g.fillRect(180+3*i + 100*(n%5), 10+3*j + 100*(n/5), 3, 3);  
        }  
    }  
}
```

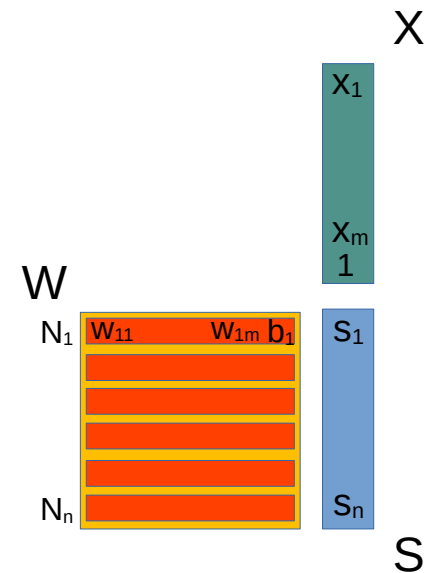
Artificial Intelligence for Robotics

- Results



Artificial Intelligence for Robotics

- Algorithmic optimization
- The Layer class call instances of Neuron
- For each neuron n : $s_n = W_n \cdot X$
- From the network perspective, weight vectors can be gathered in a matrix W and results in a vector S :
 - $S = W \cdot X$
- We will remove Class Neuron and integrate functions in the Layer class



Artificial Intelligence for Robotics

- **Algorithmic optimization**
- Duplicate project 2_SingleLayer and call the copy 2_SingleLayerV2
- In class Layer, replace the Neuron vector with
 - A matrix 'weights' for weights (the bias are in this matrix)
 - A vector 'result' to store outputs
 - A vector 'deltas' to store deltas
 - Get the learnRate and activation function from class Neuron

```
public class Layer {  
  
    public float learnRate=0.001f;  
  
    public float[][] weights;  
    public float[] results;  
    public float[] deltas;  
  
    public float sum_delta;  
  
    public Layer(int nb_input, int nb_output){  
        weights=new float[nb_output][nb_input+1]; // +1 for bias  
        results=new float[nb_output];  
        deltas=new float[nb_output];  
    }  
}
```

Artificial Intelligence for Robotics

- **Algorithmic optimization**
- Update function *compute* to perform the weighted sum of neurons
 - Be careful with indexes

```
public void compute(float[] input){  
    for (int n=0;n<results.length;n++){ // each neuron  
        results[n]=0; // reset sums  
        for (int i=0;i<input.length;i++){  
            results[n]+= weights[n][i]*input[i]; // /\ index  
        }  
        results[n]+=weights[n][input.length]; // bias  
        results[n]=activation(results[n]);  
    }  
}
```

Artificial Intelligence for Robotics

- **Algorithmic optimization**
- Update function learn to update neurons' weights and get total error

```
public void learn(float[] input, int[] output){
    // compute all deltas
    sum_delta=0;
    for (int n=0;n<deltas.length;n++){ // each neuron
        deltas[n]=output[n]-results[n];
        sum_delta+=Math.abs(deltas[n]);
    }

    // update weights
    for (int n=0;n<results.length;n++){
        for (int i=0;i<input.length;i++){
            weights[n][i]+= learnRate * deltas[n] * input[i];
        }
        weights[n][input.length]+= learnRate * deltas[n]; // bias
    }
}
```

Artificial Intelligence for Robotics

- **Algorithmic optimization**
- Correct pointers in class DisplayPanel

```
for (int j=0;j<Main.size_y;j++){  
    val=(int) (main.layer.weights[n][i+Main.size_x*j]*50)+128;
```

- Delete class Neuron (check for remaining errors)
- Test your network: results should be the same
- **Keep this project, we will reuse it !**